

Hive Console layout

Persistent top bar (always visible)

- **Mode pill:** DRY RUN / LIVE
- **Trading pill:** RUNNING / PAUSED
- **Risk banner (Kill Switch):** OK / THROTTLE / HALT + reason
- **Venue + Symbol:** e.g., Binance · ETHUSDT
- **Clock + feed status:** "Market feed: OK · Wallet: OK · Bus: OK"

This bar never scrolls away. If you hide these, you lose trust.

1) Home: "Status"

A single page that answers "what's happening?" in 10 seconds.

A. System Health cards (4 small cards)

- **Market:** last update age, WS connected, spread%, volatility hint
- **Wallet:** last update age, ETH free/locked, USDT free/locked
- **Execution:** last order latency p95, rejects (5m), last error
- **Network:** lag per topic, dropped/retried counts

B. ETH Market strip (compact, not a giant chart)

- Last / Bid / Ask / **Spread %**
- 1m volume
- Candle countdown (e.g., "5m candle closes in 01:12")

C. Portfolio strip

- Equity estimate (USD)
- Realized PnL (today)
- Unrealized PnL
- Exposure (ETH size, % equity)

D. The signature component: "Decision Chain"

A vertical timeline of the last ~20 decisions:

Each entry is a single row with 5 icons:

1. **Signal** (Strategy)

2. **Gate** (Coordinator approve/veto + reason)
3. **Order** (requested type/qty)
4. **Execution** (filled/rejected/canceled + avg price)
5. **Outcome** (PnL impact, slippage)

Example row:

- SMA_CROSS BUY 0.71 → VETO: COOLDOWN (grey)
- SMA_CROSS BUY 0.68 → APPROVE → LIMIT 0.22 ETH → FILLED @ 2350.4 → +0.18%

This is the “it’s not a toy” section.

2) Intents & Orders (the operational page)

This page is for *control*, not analysis.

A. Active intents table

Columns:

- Intent ID (clickable)
- Strategy / Action
- State (SENT / ACKED / ACTIVE / DONE)
- Attempt (A/B)
- Timer (time since placed, timeout countdown)
- Button: **Cancel intent**

B. Order details drawer (click an intent)

- Full event chain:
 - strategy signal payload
 - coordinator decision payload + veto reasons if any
 - trade request parameters
 - execution events and timestamps
- Fill breakdown:
 - avg fill price
 - fees
 - slippage%
- “Replay view”: show the market snapshot at decision time (bid/ask/spread)

3) Risk & Kill Switch (safety center)

This page should make safety feel institutional.

A. Risk state

- Current: OK/THROTTLE/HALT
- Reason
- Time since state change
- Clear conditions required to return to OK (hysteresis rules)
- **Manual HALT** button (with typed confirmation)

B. Risk meters (simple, not fancy)

- Daily PnL % vs thresholds
- Drawdown vs max
- Reject rate (5m)
- Slippage average (5m)
- Feed staleness count (hour)

C. "Why we halted"

A clear audit card:

- trigger event
- last 10 relevant events (auth failure, stale feed, loss limit breach)

4) Performance (clean, research-grade)

This page should be calm and not overfitted.

A. Equity curve (one chart)

- Equity (USD) over time
- Mark major events: HALT, large fills

B. KPI tiles

- Trades today / week
- Win rate
- Average R multiple (later)
- Fees paid

- Avg slippage
- Max drawdown

C. Session report export

- "Export last 24h" → CSV/JSON/PDF (later)
 - This is where your system starts to look sellable.
-

5) Configuration (safe knobs only)

Keep this page "boring" and guarded.

Sections

- **Trading mode:** DRY_RUN/LIVE (requires confirmation + "arming")
- **Allowed symbols:** ETHUSDT only at first
- **Max notional:** slider + numeric
- **Cooldown:** numeric
- **Market order spread guard:** numeric
- **Throttle multiplier:** numeric
- **Order type preference:** market/limit (limit recommended)

Every change produces an entry in an audit log:

- who changed
 - what changed
 - when
 - previous value → new value
-

6) Audit Log (the credibility page)

A searchable stream (filters on left):

- event type
- symbol
- severity
- source agent
- trace/intent id

Click any event → see the raw JSON envelope.

This is how you debug, and how you convince “big boys.”

7) Alerts (inbox style)

Alerts should be structured, not popups.

Each alert contains:

- severity (info/warn/high/critical)
 - source
 - reason code
 - recommended action
 - linked intent/order if relevant
-

Elegant UX principles for this system

1) “Truth over beauty”

No giant tradingview clones. Use charts sparingly and only where they explain.

2) “Decision-first”

Your unique selling point is the **explainable pipeline**. Surface it everywhere.

3) “Safe by default”

- Default PAUSED + DRY_RUN
- Two-step confirmation for LIVE/HALT/CANCEL ALL
- Always show current mode

4) “Operator calm”

Muted palette, clear hierarchy, minimal animation. Alerts are controlled, not chaotic.

5) “Everything links back to an intent”

If a user clicks anything, they can trace it back to:

- the signal
- the gate
- the order
- the fill
- the outcome

That traceability is what makes it feel like a platform.

If you want a concrete visual

I can describe a clean wireframe for each screen (layout + components), or generate a Streamlit/React skeleton that matches this “Hive Console” structure.

provide wireframe

Below is a **clean, implementation-ready wireframe** for the **Hive Console**.

This is not visual fluff — it’s a **layout spec** you can hand to yourself (or a dev) and build in Streamlit, React, or even a terminal UI.

I’ll give you:

1. Overall layout grid
2. Wireframe per screen (ASCII + component list)
3. Interaction notes (what updates what)



HIVE CONSOLE — WIREFRAME SPEC

GLOBAL LAYOUT (ALL SCREENS)

HIVENANCE 		[DRY RUN]	[RUNNING]	[RISK: OK]	ETHUSDT • BINANCE
Market: OK • Wallet: OK • Exec: OK • Bus: OK		 12:34:56			
Sidebar	Main Content Area				
▸ Status					
▸ Intents					
▸ Risk					
▸ Performance					
▸ Config					

► Audit

Notes

- Top bar is fixed (never scrolls)
- Status pills change color:
 - OK = green
 - THROTTLE = amber
 - HALT = red
- Clicking any pill opens its detail page

1 STATUS (HOME)

Purpose

“Is the system healthy, and what just happened?”

WIREFRAME

SYSTEM HEALTH			
Market	Wallet	Exec	Network
OK	OK	OK	OK
age 1s	age 3s	p95 420ms	lag 0

ETH MARKET SNAPSHOT			
Last: 2350.42	Bid: 2350.10	Ask: 2350.60	Spread: 0.021%
Volume (1m): 1,243 ETH		Candle closes in: 01:12	

PORTFOLIO		
Equity: \$4,120.55	Realized PnL: -\$18.20	Unrealized: +\$6.10
Exposure: 0.42 ETH (24% equity)		

DECISION CHAIN (LATEST)				
Time	Strategy	Signal	Gate	Execution
10:32:05	SMA_CROSS	BUY (0.71)	VETO (COOLDOWN)	—
10:35:00	SMA_CROSS	BUY (0.68)	APPROVE	FILLED
10:35:01	—	—	—	@2350.4

Interactions

- Clicking a row → opens Intent Detail Drawer
- Clicking Market/Wallet cards → jumps to relevant page

2 INTENTS & ORDERS

Purpose

“What is active right now, and can I intervene?”

WIREFRAME

ACTIVE INTENTS					
Intent ID	State	Attempt	Qty	Timer	Action
intent-...-001	ACTIVE	A	0.22	00:04	[Cancel]
intent-...-002	DONE	—	0.18	—	View

INTENT DETAIL (SLIDE-OUT DRAWER)	
Intent:	intent-...-001
Strategy:	SMA_CROSS
Signal:	BUY (confidence 0.68)
Coordinator:	APPROVED
Reason:	POLICY_OK
Order:	LIMIT • 0.22 ETH
Client Order ID:	hive-...-A
Exchange Order ID:	BN-982734
Execution:	PARTIAL (0.12 / 0.22)
Avg Price:	2350.4
Fees:	\$0.21

```
[Cancel Order]    [View Market Snapshot]
```

3 RISK & KILL SWITCH

Purpose

"Is the system safe right now, and why?"

WIREFRAME

```
----- RISK STATE -----
CURRENT: THROTTLE
Reason: REJECT_SPIKE
Active since: 00:12:34
Clear conditions:
- Rejects < 2 for 10 minutes
- Market feed stable

[HALT NOW] (type HALT to confirm)
```

```
----- RISK METERS -----
Daily PnL:   -0.44%  [██████████]
Drawdown:    1.12%  [██████████]
Rejects (5m): 6      [██████████]
Slippage:     0.28%  [██████████]
```

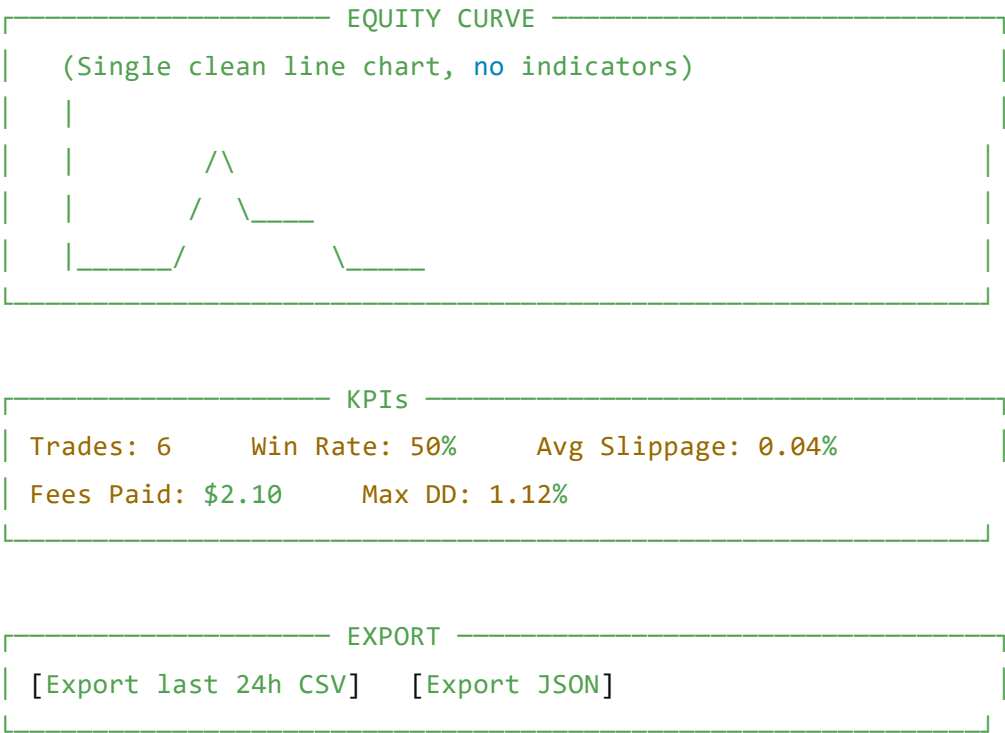
```
----- WHY WE THROTTLED -----
10:33:12 Reject spike detected (6 in 5m)
10:33:15 Market spread elevated
10:33:16 Kill switch set to THROTTLE
```

4 PERFORMANCE

Purpose

“Is this actually working?”

WIREFRAME

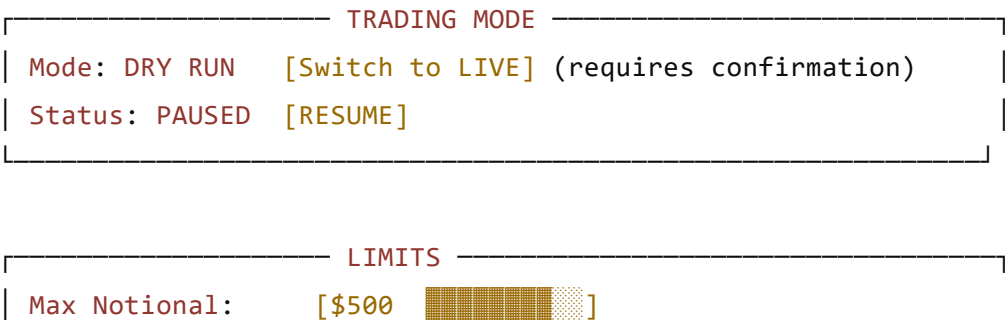


5 CONFIG (SAFE)

Purpose

“Tune without shooting yourself in the foot.”

WIREFRAME



Cooldown (sec):	[180]
Market Spread %:	[0.15]
Throttle Mult.:	[0.40]

AUDIT	
Last change:	max_notional 300 → 500 (byron @ 10:21)

6 AUDIT LOG

Purpose

"Prove what happened."

WIREFRAME

FILTERS			
Type ▾	Source ▾	Severity ▾	Intent ID ▾

EVENTS	
10:35:01	trade.execution FILLED intent-...-001
10:35:00	coordinator.decision APPROVE intent-...-001
10:32:05	strategy.signal BUY SMA_CROSS
10:31:58	kill.check THROTTLE REJECT_SPIKE

(Click any row → raw JSON envelope)

Why this UI is elegant

- **Zero guesswork:** everything traces to an intent
- **Safety-first:** risk state is always visible
- **Explainable:** vetoes are first-class citizens
- **Sellable:** looks like a system, not a bot

If you want next:

- I can turn this into a **Streamlit app skeleton**
- Or produce a **React component tree**
- Or design a **dark/light theme palette** that fits the bee/hive aesthetic 🐝

React component tree please

Hivenance Hive Console – React Component Tree (Implementation-Oriented)

```

===== <App>
<Providers> <ThemeProvider /> // light/dark, brand tokens <AuthProvider />
// operator session + permissions <BuzzBusProvider /> // websocket/redis-
bridge client <QueryClientProvider /> // react-query for cached selectors
<ToastProvider /> // alert toasts (optional) <AppShell> // global layout:
topbar + sidebar + main <TopBar> <BrandMark /> // HIVENANCE + bee
<ModePills> <ModePill kind="DRY_RUN|LIVE" /> <ModePill kind="RUNNING|PAUSED"
/> <RiskPill state="OK|THROTTLE|HALT" /> </ModePills> <ContextPills>
<VenueSymbolPill venue="BINANCE" symbol="ETHUSDT" /> <HealthPills>
<HealthPill label="Market" state="OK|DEGRADED|FAIL" /> <HealthPill
label="Wallet" state="OK|DEGRADED|FAIL" /> <HealthPill label="Exec"
state="OK|DEGRADED|FAIL" /> <HealthPill label="Bus" state="OK|DEGRADED|FAIL"
/> </HealthPills> </ContextPills> <Clock /> <UserMenu> <UserAvatar />
<UserDropdown> <MenuItem>Profile</MenuItem> <MenuItem>API Status</MenuItem>
<MenuItem>Logout</MenuItem> </UserDropdown> </UserMenu> </TopBar>
<SidebarNav> <NavItem route="/status" icon="Gauge">Status</NavItem> <NavItem
route="/intents" icon="Layers">Intents & Orders</NavItem> <NavItem
route="/risk" icon="Shield">Risk</NavItem> <NavItem route="/performance"
icon="LineChart">Performance</NavItem> <NavItem route="/config"
icon="Sliders">Config</NavItem> <NavItem route="/audit"
icon="List">Audit</NavItem> </SidebarNav> <Main> <RouterOutlet> <StatusPage
/> <IntentsPage /> <RiskPage /> <PerformancePage /> <ConfigPage />
<AuditPage /> </RouterOutlet> </Main> <GlobalOverlays> <IntentDetailDrawer
/> // opens from Decision Chain or Intents table <ConfirmModal /> // typed
confirmations for HALT/LIVE/CANCEL_ALL <ToastStack /> // alerts
</GlobalOverlays> </AppShell> </Providers> </App> State / Data Layer
(recommended structure) -----
<BuzzBusProvider> <BuzzBusClient /> // WS client, reconnection,
subscriptions <EventReducerStore /> // global state via reducer (or
Zustand/Redux) <Selectors /> // derived state: staleness, exposure, etc.
</BuzzBusProvider> Suggested store slices: - system: { mode, paused, dryRun,
venue, symbols } - health: { market, wallet, exec, bus, lastUpdatedTs } -
market: { bySymbol: { last, bid, ask, spreadPct, candleCountdown, ts } } -
wallet: { balances, equityUsdEst, ts } - risk: { state, reason, ts,
hysteresisRules } - decisions: { recentDecisionChain[] } // joined signal +
decision + execution - intents: { byId, activeIds, historyIds } - orders: {

```

```

byClientIdOrderId, byOrderId } - performance: { snapshots, equityCurve[] } -
audit: { filters, events[] } - ui: { drawerOpen, selectedIntentId,
modalState, toasts[] } Pages (component detail) -----
<StatusPage> <PageHeader title="Status" /> <Grid cols=12 gap=...>
<SystemHealthCards colSpan=12> <HealthCard type="Market" /> <HealthCard
type="Wallet" /> <HealthCard type="Execution" /> <HealthCard type="Network"
/> </SystemHealthCards> <MarketSnapshotPanel colSpan=12> <PriceStrip />
<SpreadChip /> <VolumeChip /> <CandleCountdown /> <MiniSparkline /> //
optional, tiny </MarketSnapshotPanel> <PortfolioPanel colSpan=12>
<EquityTile /> <RealizedPnlTile /> <UnrealizedPnlTile /> <ExposureTile />
</PortfolioPanel> <DecisionChainPanel colSpan=12> <DecisionChainTable>
<DecisionRow onClick=openIntentDrawer /> </DecisionChainTable>
</DecisionChainPanel> </Grid> </StatusPage> <IntentsPage> <PageHeader
title="Intents & Orders" /> <SplitLayout> <IntentsTable> <IntentRow actions>
<Button>View</Button> <Button variant="danger">Cancel</Button> </IntentRow>
</IntentsTable> <OrdersTable> // optional second table <OrderRow />
</OrdersTable> </SplitLayout> </IntentsPage> <RiskPage> <PageHeader
title="Risk" /> <Grid cols=12> <RiskStatePanel colSpan=12> <RiskBanner />
<RiskReason /> <RiskSince /> <ClearConditionsList /> <DangerZone>
<HaltNowButton opens ConfirmModal typed="HALT" /> <CancelAllButton opens
ConfirmModal typed="CANCEL" /> </DangerZone> </RiskStatePanel>
<RiskMetersPanel colSpan=12> <Meter label="Daily PnL %" /> <Meter
label="Drawdown %" /> <Meter label="Rejects (5m)" /> <Meter label="Avg
Slippage (5m)" /> <Meter label="Feed Stale Events" /> </RiskMetersPanel>
<RiskTimelinePanel colSpan=12> <RiskEventList /> </RiskTimelinePanel>
</Grid> </RiskPage> <PerformancePage> <PageHeader title="Performance" />
<Grid cols=12> <EquityChartPanel colSpan=12> <EquityLineChart />
</EquityChartPanel> <KpiTilesPanel colSpan=12> <KpiTile label="Trades" />
<KpiTile label="Win Rate" /> <KpiTile label="Max DD" /> <KpiTile label="Fees
Paid" /> <KpiTile label="Avg Slippage" /> </KpiTilesPanel> <ExportPanel
colSpan=12> <ExportButton format="CSV" /> <ExportButton format="JSON" />
</ExportPanel> </Grid> </PerformancePage> <ConfigPage> <PageHeader
title="Config" /> <Grid cols=12> <TradingModePanel colSpan=12> <ToggleDryRun
/> // guarded <TogglePaused /> <GoLiveButton opens ConfirmModal typed="ARM
LIVE" /> </TradingModePanel> <LimitsPanel colSpan=12> <SliderField
label="Max Notional" /> <NumberField label="Cooldown (sec)" /> <NumberField
label="Market Spread Guard (%)" /> <NumberField label="Throttle Multiplier"
/> <SelectField label="Order Type Preference" /> </LimitsPanel>
<ConfigAuditPanel colSpan=12> <ConfigChangeList /> </ConfigAuditPanel>
</Grid> </ConfigPage> <AuditPage> <PageHeader title="Audit" /> <Grid
cols=12> <AuditFilters colSpan=12> <FilterSelect label="Type" />
<FilterSelect label="Source" /> <FilterSelect label="Severity" />
<FilterInput label="Intent ID / Trace" /> <DateRangePicker />
</AuditFilters> <AuditTable colSpan=12> <AuditRow onClick=openRawJsonModal
/> </AuditTable> <RawJsonModal /> // shows selected event envelope </Grid>
</AuditPage> Shared Components (reusable) -----
<PageHeader />, <Grid />, <Panel />, <Card />, <Chip />, <Badge /> -
<StalenessIndicator ts=... maxAge=... /> - <DataFreshnessDot /> - <Icon />
(lucide-react) - <ConfirmModal typed="HALT|CANCEL|ARM LIVE" /> - <IntentLink
intentId="..." /> // opens drawer - <CopyButton value="intent-..." /> -

```

```

<JsonViewer data={...} /> Command / Control Components (UI → Security →
Coordinator) -----
<CommandButton> props: - commandName - payload - requiresConfirmation:
boolean - confirmationPhrase: string - severity: "normal|danger"
Implementation: - calls useCommand() hook: - emits buzz.system.command with
auth token - waits for buzz.ui.ack / or displays error toast Hooks /
Services (recommended) -----
useBuzzSubscribe(topic, handler) useAppSelectors() // derived state
useIntentActions() // cancel intent, view details useCommand() //
pause/resume/halt/config update useStaleness(ts, maxAgeSec)
useDecisionChainBuilder() // join signal/decision/execution by intent_id
usePersistedUIState() // last page, filters, theme Routes (minimal) -----
----- /status /intents /risk /performance /config /audit

```

If you want, I can also provide:

- a *data contract* for the "Decision Chain builder" (how to join events by `intent_id` / `trace_id`), and
- a minimal `AppShell` + `StatusPage` React skeleton (no backend assumptions).

sure go ahead with the data contract

Below is a **data contract** for building the UI's **Decision Chain** (Signal → Gate → Order → Execution → Outcome) in a way that's robust to **out-of-order events**, **retries**, and **at-least-once delivery**.

I'll define:

1. Canonical IDs + linkage rules
2. Required fields per event type
3. A "DecisionChainItem" UI model
4. Joining algorithm (deterministic)
5. Edge cases (retries/partials/missing events)

1) Canonical IDs and linkage rules

Required identifiers

- `intent_id`: created by Coordinator when it approves a trade intent.
- `client_order_id`: created by Coordinator per attempt (...-A , ...-B).

- `order_id` : assigned by exchange (optional early but ideal).

Trace linkage

Every message should include:

- `buzz.trace_id = intent_id` **when available**

Rule:

- Strategy signals may not have `intent_id` yet (they occur before approval).
- Once Coordinator approves, all downstream messages must carry `intent_id`.

Recommended: add `signal_id`

Have Strategy emit a unique `signal_id`. Coordinator includes it when creating the intent so you can link "the signal that caused the intent".

2) Required fields per message type (minimum contract)

A) `buzz.strategy.signal`

Used to show "Signal" in the chain.

Required

- `payload.symbol`
- `payload.action` (`BUY|SELL|HOLD`)
- `payload.strategy` (e.g., `SMA_CROSS`)
- `payload.confidence` (0–1) *or omit if not using*
- `buzz.id` and `buzz.ts`

Recommended

- `payload.signal_id` (unique)
- `payload.features_snapshot` (tiny: fast/slow SMA values, etc.)

Example:

```
{ "buzz": { "id": "...", "type": "buzz.strategy.signal", "source": "STRATEGY",
"ts": 1730000000000 }, "payload": { "signal_id": "sig-1730000000-eth-0009",
"symbol": "ETHUSDT", "action": "BUY", "strategy": "SMA_CROSS", "confidence":
0.68, "features_snapshot": { "sma_fast": 2348.1, "sma_slow": 2346.2 } } }
```

B) `buzz.coordinator.decision`

Used to show "Gate" outcome (APPROVE/VETO + reason).

Required

- `payload.intent_id` (even if veto; generate a "proposed intent id")
- `payload.symbol` , `payload.action`
- `payload.decision` (APPROVE|VETO)
- `payload.reason` (reason code)
- `buzz.ts`

Recommended

- `payload.signal_id` (to link back to strategy signal)
- `payload.policy_snapshot` (key values: cooldown, max_notional, spread_guard)
- `payload.kill_state_at_decision`

Example:

```
{ "buzz": { "id": "...", "type": "buzz.coordinator.decision",
"source": "COORDINATOR", "ts": 1730000000100, "trace_id": "intent-1730000000-ethusdt-0001" }, "payload": { "intent_id": "intent-1730000000-ethusdt-0001",
"signal_id": "sig-1730000000-eth-0009", "symbol": "ETHUSDT", "action": "BUY",
"decision": "APPROVE", "reason": "POLICY_OK", "kill_state_at_decision": "OK",
"policy_snapshot": { "cooldown_sec": 180, "max_notional_usd": 500,
"spread_guard_pct": 0.15 } } }
```

C) `buzz.trade.request`

Used to show "Order request" parameters.

Required

- `payload.intent_id`
- `payload.client_order_id`
- `payload.venue` , `payload.symbol` , `payload.side`
- `payload.order_type` , `payload.qty`
- `buzz.ts`

Recommended

- `payload.attempt` (A|B|C)
- `payload.expire_after_sec`

- `payload.max_slippage_pct`
-

D) `buzz.trade.execution`

Used to show "Execution" status progression and final outcome.

Required

- `payload.intent_id`
- `payload.client_order_id`
- `payload.status`
- `buzz.ts`

Recommended

- `payload.order_id`
- `payload.filled_qty` , `payload.avg_price` , `payload.fees`
- `payload.slippage_pct` , `payload.latency_ms`
- `payload.error.class` , `payload.error.message`

Statuses you must support:

- `RECEIVED` , `PRECHECK_FAILED` , `PLACED` , `PARTIAL` , `FILLED` , `CANCELED` , `REJECTED` , `EXPIRED` , `ERROR`
-

E) Optional: `buzz.intent.state`

Useful for chain completeness and UI "current state" display.

Required

- `payload.intent_id`
- `payload.state`
- `buzz.ts`

Recommended: attempt and order ids.

3) UI Model: `DecisionChainItem`

This is the normalized object your UI renders per "row" in the Decision Chain.

```

type DecisionChainItem = { intentId: string; symbol: string; side: "BUY" |
"SELL"; strategy?: string; // Signal signal?: { signalId?: string; ts:
number; action: "BUY" | "SELL" | "HOLD"; confidence?: number;
featuresSnapshot?: Record<string, number | string>; }; // Gate gate: { ts:
number; decision: "APPROVE" | "VETO"; reason: string; killState?: "OK" |
"THROTTLE" | "HALT"; notes?: string; policySnapshot?: Record<string, number
| string>; }; // Order attempt(s) attempts: Array<{ attemptId: string; //
usually client_order_id venue: string; orderType: "MARKET" | "LIMIT"; qty:
number; requestedTs: number; expireAfterSec?: number; // Execution timeline
exec: { lastStatus: string; statuses: Array<{ ts: number; status: string;
details?: any }>; orderId?: string; filledQty?: number; avgPrice?: number;
fees?: number; slippagePct?: number; latencyMs?: number; error?: { class:
string; message?: string }; finalTs?: number; finalOutcome?:
"FILLED"|"CANCELED"|"REJECTED"|"EXPIRED"|"ERROR"|"PRECHECK_FAILED"; }; }>;
// Outcome summary (best effort) outcome?: { realizedPnlUsd?: number; //
from Performance agent if available markToMarketPnlUsd?: number; // optional
}; // UI status helpers chainState: | "VETOED" | "PENDING_REQUEST" |
"PLACED" | "PARTIAL" | "FILLED" | "FAILED" | "CANCELED"; };

```

4) Joining algorithm (deterministic)

A) Primary join key: intent_id

The Decision Chain is fundamentally **intent-centric**.

Maintain a map:

- `intentMap[intent_id] => DecisionChainItem`

B) Secondary join key: signal_id

For linking Strategy → Coordinator:

- `signalMap[signal_id] => strategy.signal`
- Coordinator.decision references `signal_id` if possible

C) Execution attempts

Within an intent:

- `attemptKey = client_order_id`
 - `attempts` array is keyed/indexed by `client_order_id`
-

5) Event → Chain update rules

When `strategy.signal` arrives

- Store in `signalMap` by `signal_id` (or `buzz.id` if no `signal_id`)
- Optionally show in a "Signals (unapproved)" feed, but **not** in Decision Chain yet unless it's linked to an intent.

When `coordinator.decision` arrives

- Create or update `DecisionChainItem` by `intent_id`
- Fill `gate` block
- If it includes `signal_id`, attach the signal from `signalMap`
- If `decision = VETO`:
 - set `chainState = "VETOED"`
 - `attempts` stays empty

When `trade.request` arrives

- Find chain by `intent_id`
- Upsert attempt by `client_order_id`
- Populate `attempt.request` fields
- set `chainState = "PENDING_REQUEST"` (unless already placed)

When `trade.execution` arrives

- Find chain by `intent_id`
- Upsert attempt by `client_order_id`
- Append status transition to `attempt.exec.statuses` (dedupe by (`status`, `ts`) or message id)
- Update `attempt.exec.lastStatus`

Then set `chainState`:

- `PLACED` → "PLACED"
- `PARTIAL` → "PARTIAL"
- `FILLED` → "FILLED"
- `CANCELED` → "CANCELED" (unless another attempt is active)
- `REJECTED/ERROR/PRECHECK_FAILED/EXPIRED` → "FAILED" (unless retry attempt exists)

When `performance.snapshot` arrives (optional)

- Match by symbol and/or last filled intent (or store a mapping in Data Store)

- Update outcome summary if you can associate PnL to intent (nice-to-have, not required for Phase 1)
-

6) Edge cases (must handle cleanly)

A) Out-of-order events

Execution may arrive before request (rare but possible via replay).

Solution:

- Upsert attempts even if request missing; mark request fields as unknown until request arrives.

B) Retries (attempt B/C)

- attempts array contains A then B then C
- chainState should reflect the **most recent active attempt**
- UI should show a small badge: "Retry B"

C) Partial fills + cancel

- If PARTIAL then CANCELED:
 - show finalOutcome = CANCELED
 - but keep filledQty and avgPrice visible
 - chainState could be "CANCELED (PARTIAL)" as a label

D) Vetoed decisions

- A vetoed intent should still render as a row:
 - signal + veto reason
 - no attempts

This is key to explainability.

E) Missing signal link

If Coordinator doesn't include signal_id:

- DecisionChain still works from decision onward
- Signal cell can show "(unlinked)" or show nothing

Strongly recommended to include signal_id to avoid this.

7) Required "reason codes" (contract)

To keep UI consistent, standardize reason codes:

Coordinator veto reasons (examples):

- UI_PAUSED
- KILL_SWITCH_HALT
- COOLDOWN_ACTIVE
- STALE_SIGNAL
- WALLET_STALE
- MARKET_STALE_OR_DEGRADED
- SPREAD_TOO_WIDE
- INSUFFICIENT_FUNDS
- SYMBOL_NOT_ALLOWED

Execution failure reasons:

- RATE_LIMIT
- AUTH_FAILED
- MIN_NOTIONAL
- INVALID_SYMBOL
- INSUFFICIENT_FUNDS
- NETWORK_ERROR
- EXCHANGE_REJECT
- UNKNOWN

These become filterable in the Audit page too.

8) Minimal contract checklist (so the UI works)

If you implement only these fields, the Decision Chain will be solid:

- Coordinator decision always
includes `intent_id` , `decision` , `reason` , `symbol` , `action`
 - Trade request includes `intent_id` , `client_order_id` , `order_type` , `qty`
 - Trade execution includes `intent_id` , `client_order_id` , `status` (+
optional `avg_price` , `fees` , `slippage_pct`)
 - Add `signal_id` if you want the full "signal → gate" link
-

If you want next, I can provide a **TypeScript interface file** (`buzz-types.ts`) containing these event payload typings + a `DecisionChainBuilder` reducer that consumes events and emits the UI-ready chain list.

sure, provide typescript interface

```
// buzz-types.ts // Hivenance / SwarmOS – Buzz Protocol TypeScript
Interfaces // ===== //
Purpose: // - Define message envelope + key event payloads // - Provide a
UI-ready Decision Chain model // - Provide reason-code enums (optional but
recommended) // // Notes: // - Keep "payload" minimal and stable across
agents // - Add fields over time without breaking existing consumers /* ----
----- * Core Envelope * ----- */
export type BuzzEventType = | "buzz.strategy.signal" |
"buzz.coordinator.decision" | "buzz.trade.request" | "buzz.trade.execution"
| "buzz.intent.state" | "buzz.wallet.balance" | "buzz.wallet.activity" |
"buzz.market.data" | "buzz.market.candle" | "buzz.market.health" |
"buzz.analytics.snapshot" | "buzz.performance.snapshot" | "buzz.kill.check"
| "buzz.kill.trigger" | "buzz.system.command" |
"buzz.system.command.authorized" | "buzz.security.event" | "buzz.net.health"
| "buzz.exec.health" | "buzz.store.query" | "buzz.store.result"; export type
BuzzSource = | "MARKET_DATA" | "WALLET_MONITOR" | "STRATEGY" | "COORDINATOR"
| "EXECUTION" | "ANALYTICS" | "KILL_SWITCH" | "UI" | "SECURITY" | "NETWORK"
| "DATA_STORE" | "PERFORMANCE" | "UNKNOWN"; export interface BuzzMeta { /**
Unique event id (ULID/UUID). Required for dedupe. */ id: string; /** Buzz
event type */ type: BuzzEventType; /** Event producer */ source: BuzzSource;
/** Milliseconds epoch */ ts: number; /** Optional trace id; set to
intent_id when known */ trace_id?: string; /** Optional schema version for
payload evolution */ schema_v?: number; } export interface
BuzzMessage<TPayload> = unknown { buzz: BuzzMeta; payload: TPayload; } /* --
----- * Shared primitives * ----- */
export type Venue = "binance" | "kraken" | "coinbase" | "bybit" | "dex" |
"unknown"; export type SymbolPair = string; // e.g., "ETHUSDT", "BTCUSD"
export type Side = "BUY" | "SELL"; export type TradeAction = Side | "HOLD";
export type OrderType = "MARKET" | "LIMIT" | "STOP" | "STOP_LIMIT"; export
type KillState = "OK" | "THROTTLE" | "HALT"; export type HealthState = "OK"
| "DEGRADED" | "FAIL" | "UNKNOWN"; export type Severity = "INFO" | "WARN" |
"HIGH" | "CRITICAL"; /* ----- * Reason codes
(recommended) * ----- */ // Coordinator veto/approve
reasons export type CoordinatorReason = | "POLICY_OK" | "UI_PAUSED" |
"KILL_SWITCH_HALT" | "COOLDOWN_ACTIVE" | "STALE_SIGNAL" | "WALLET_STALE" |
"MARKET_STALE_OR_DEGRADED" | "SPREAD_TOO_WIDE" | "INSUFFICIENT_FUNDS" |
"SYMBOL_NOT_ALLOWED" | "NO_PRICE" | "NO_EQUITY_ESTIMATE" |
"QTY_COMPUTE_FAILED" | "UNKNOWN"; // Execution error classes (examples)
export type ExecutionErrorClass = | "RATE_LIMIT" | "AUTH_FAILED" |
```

```

"MIN_NOTIONAL" | "INVALID_SYMBOL" | "INSUFFICIENT_FUNDS" | "NETWORK_ERROR" |
"EXCHANGE_REJECT" | "TIMEOUT" | "UNKNOWN"; /* ----- *
Strategy (Agent 3) * ----- */ export interface
StrategySignalPayload { signal_id: string; symbol: SymbolPair; action:
TradeAction; // BUY/SELL/HOLD strategy: string; // e.g., "SMA_CROSS"
confidence?: number; // 0..1 timeframe?: string; // "1m" | "5m" etc.
features_snapshot?: Record<string, number | string | boolean>; } export type
StrategySignalMessage = BuzzMessage<StrategySignalPayload>; /* -----
----- * Coordinator (Agent 5) * ----- */ export
interface CoordinatorDecisionPayload { intent_id: string; signal_id?:
string; symbol: SymbolPair; action: Side; // HOLD signals shouldn't reach
coordinator decision decision: "APPROVE" | "VETO"; reason: CoordinatorReason
| string; notes?: string; kill_state_at_decision?: KillState;
policy_snapshot?: Record<string, number | string | boolean>; } export type
CoordinatorDecisionMessage = BuzzMessage<CoordinatorDecisionPayload>; /* ---
----- * Trade Request (Coordinator -> Execution) * -----
----- */ export interface TradeRequestPayload { intent_id:
string; client_order_id: string; venue: Venue; symbol: SymbolPair; side:
Side; order_type: OrderType; qty: number; // optional order params price?:
number; // for LIMIT time_in_force?: "GTC" | "IOC" | "FOK";
max_slippage_pct?: number; expire_after_sec?: number; origin_strategy?:
string; attempt?: string; // "A"/"B"/... } export type TradeRequestMessage =
BuzzMessage<TradeRequestPayload>; /* ----- * Trade
Execution (Execution Agent -> Bus) * ----- */ export
type ExecutionStatus = | "RECEIVED" | "PRECHECK_FAILED" | "PLACED" |
"PARTIAL" | "FILLED" | "CANCELED" | "REJECTED" | "EXPIRED" | "ERROR"; export
interface TradeExecutionError { class: ExecutionErrorClass | string;
message?: string; raw_code?: string | number; } export interface
TradeExecutionPayload { intent_id: string; client_order_id: string; venue:
Venue; symbol: SymbolPair; status: ExecutionStatus; // exchange linkage
order_id?: string; // fills/metrics filled_qty?: number; avg_price?: number;
fee_usd?: number; fee_asset?: string; fee_amount?: number; slippage_pct?:
number; latency_ms?: number; // misc error?: TradeExecutionError; } export
type TradeExecutionMessage = BuzzMessage<TradeExecutionPayload>; /* -----
----- * Intent State (optional but useful) * -----
----- */ export type IntentLifecycleState = | "INTENT_NEW" |
"INTENT_VALIDATED" | "INTENT_SENT" | "INTENT_ACKED" | "INTENT_ACTIVE" |
"INTENT_DONE"; export interface IntentStatePayload { intent_id: string;
symbol: SymbolPair; state: IntentLifecycleState; final_outcome?: "FILLED" |
"CANCELED" | "REJECTED" | "EXPIRED" | "ERROR" | "PRECHECK_FAILED" |
"ABORTED"; client_order_id?: string; order_id?: string; reason?: string; }
export type IntentStateMessage = BuzzMessage<IntentStatePayload>; /* -----
----- * Wallet (Agent 1) * ----- */
export interface WalletBalanceItem { asset: string; // "ETH", "USDT" free:
number; locked: number; } export interface WalletBalancePayload {
wallet_type: "cex" | "evm" | "unknown"; venue?: Venue; // "binance" etc when
cex address?: string; // when evm balances: WalletBalanceItem[];
equity_usd_est?: number; } export type WalletBalanceMessage =
BuzzMessage<WalletBalancePayload>; export interface WalletActivityPayload {
wallet_type: "cex" | "evm" | "unknown"; venue?: Venue; address?: string;

```

```

activity_type: "DEPOSIT" | "WITHDRAWAL" | "TRANSFER" | "SWAP" | "UNKNOWN";
asset?: string; amount?: number; tx_hash?: string; } export type
WalletActivityMessage = BuzzMessage<WalletActivityPayload>; /* -----
----- * Market (Agent 2) * ----- */ export
interface MarketDataPayload { venue: Venue; symbol: SymbolPair; last?:
number; bid?: number; ask?: number; spread_pct?: number; volume_1m?: number;
} export type MarketDataMessage = BuzzMessage<MarketDataPayload>; export
interface MarketCandlePayload { venue: Venue; symbol: SymbolPair; timeframe:
string; // "1m", "5m", ... open_ts: number; // ms close_ts: number; // ms o:
number; h: number; l: number; c: number; v: number; } export type
MarketCandleMessage = BuzzMessage<MarketCandlePayload>; export interface
MarketHealthPayload { venue: Venue; symbol: SymbolPair; state: HealthState;
reason?: string; } export type MarketHealthMessage =
BuzzMessage<MarketHealthPayload>; /* ----- * Kill
Switch (Agent 7) * ----- */ export interface
KillCheckPayload { risk_state: KillState; reason: string; metrics?:
Record<string, number | string | boolean>; } export type KillCheckMessage =
BuzzMessage<KillCheckPayload>; export interface KillTriggerPayload {
risk_state: "HALT"; reason: string; action?: "CANCEL_ALL" | "PAUSE" |
"NONE"; } export type KillTriggerMessage = BuzzMessage<KillTriggerPayload>;
/* ----- * UI Commands (Agent 8) * -----
----- */ export type SystemCommand = | "PAUSE_TRADING" | "RESUME_TRADING" |
"SET_DRY_RUN" | "CANCEL_ALL" | "HALT_NOW" | "CANCEL_INTENT" |
"SET_POLICY_LIMITS" | "ARM_LIVE"; export interface SystemCommandPayload {
command: SystemCommand; scope: "GLOBAL" | "SYMBOL"; symbol?: SymbolPair; //
params for specific commands dry_run?: boolean; // for SET_DRY_RUN
intent_id?: string; // for CANCEL_INTENT policy_update?: Partial<{
max_notional_usd: number; cooldown_sec: number; spread_guard_pct: number;
throttle_multiplier: number; allowed_symbols: SymbolPair[];
order_type_preference: OrderType; }>; requested_by?: string; reason?:
string; auth?: { token: string }; } export type SystemCommandMessage =
BuzzMessage<SystemCommandPayload>; export interface AuthorizedCommandPayload
{ command: SystemCommand; scope: "GLOBAL" | "SYMBOL"; symbol?: SymbolPair;
intent_id?: string; policy_update?: SystemCommandPayload["policy_update"];
requested_by?: string; } export type AuthorizedCommandMessage =
BuzzMessage<AuthorizedCommandPayload>; /* ----- *
Security Events (Agent 12) * ----- */ export interface
SecurityEventPayload { severity: Severity; type: | "UNAUTHORIZED_COMMAND" |
"MISSING_SECRET" | "EXCHANGE_KEY_OVERPERMISSIONED" | "EXCHANGE_AUTH_FAILURE"
| "CONFIG_TAMPER" | "WITHDRAWAL_DETECTED" | "UNKNOWN"; details?:
Record<string, number | string | boolean>; recommended_action?: "HALT" |
"THROTTLE" | "NONE"; } export type SecurityEventMessage =
BuzzMessage<SecurityEventPayload>; /* ----- * Network
health (Agent 9) * ----- */ export interface
NetHealthTopicStat { lag: number; pending?: number; last_ts?: number; }
export interface NetHealthPayload { bus: "redis_streams" | "rabbitmq" |
"kafka" | "unknown"; state: HealthState; topics?: Record<string,
NetHealthTopicStat>; } export type NetHealthMessage =
BuzzMessage<NetHealthPayload>; /* ----- * Execution
health (Agent 4) * ----- */ export interface

```

```

ExecHealthPayload { venue: Venue; state: HealthState; reason?: string;
rate_limit_remaining?: number; auth_ok?: boolean; last_error?: string; }
export type ExecHealthMessage = BuzzMessage<ExecHealthPayload>; /* -----
----- * Analytics snapshot (Agent 6) * -----
- */ export interface AnalyticsSnapshotPayload { window_sec: number;
symbol?: SymbolPair; trades?: number; filled?: number; rejected?: number;
avg_slippage_pct?: number; p95_latency_ms?: number; market_stale_events?:
number; wallet_stale_events?: number; } export type AnalyticsSnapshotMessage
= BuzzMessage<AnalyticsSnapshotPayload>; /* ----- *
Performance snapshot (Agent 11) * ----- */ export
interface PerformanceSnapshotPayload { symbol?: SymbolPair; window: "5m" |
"1h" | "1d" | "all"; equity_usd_est?: number; realized_pnl_usd?: number;
unrealized_pnl_usd?: number; pnl_pct?: number; max_drawdown_pct?: number;
win_rate?: number; trades?: number; avg_fee_usd?: number; } export type
PerformanceSnapshotMessage = BuzzMessage<PerformanceSnapshotPayload>; /*
===== * UI Decision
Chain Model * ===== */
export type ChainState = | "VETOED" | "PENDING_REQUEST" | "PLACED" |
"PARTIAL" | "FILLED" | "FAILED" | "CANCELED"; export interface
DecisionChainAttemptExecStatus { ts: number; status: ExecutionStatus;
details?: Record<string, any>; } export interface DecisionChainAttempt {
attemptId: string; // client_order_id venue: Venue; orderType: OrderType;
qty: number; requestedTs: number; expireAfterSec?: number; exec: {
lastStatus: ExecutionStatus; statuses: DecisionChainAttemptExecStatus[];
orderId?: string; filledQty?: number; avgPrice?: number; feesUsd?: number;
slippagePct?: number; latencyMs?: number; error?: TradeExecutionError;
finalTs?: number; finalOutcome?: "FILLED" | "CANCELED" | "REJECTED" |
"EXPIRED" | "ERROR" | "PRECHECK_FAILED"; }; } export interface
DecisionChainItem { intentId: string; symbol: SymbolPair; side: Side;
strategy?: string; signal?: { signalId?: string; ts: number; action:
TradeAction; confidence?: number; featuresSnapshot?: Record<string, number |
string | boolean>; }; gate: { ts: number; decision: "APPROVE" | "VETO";
reason: string; killState?: KillState; notes?: string; policySnapshot?:
Record<string, number | string | boolean>; }; attempts:
DecisionChainAttempt[]; outcome?: { realizedPnlUsd?: number;
markToMarketPnlUsd?: number; }; chainState: ChainState; } /* -----
----- * Convenience union (all known message types) * -----
----- */ export type AnyBuzzMessage = | StrategySignalMessage |
CoordinatorDecisionMessage | TradeRequestMessage | TradeExecutionMessage |
IntentStateMessage | WalletBalanceMessage | WalletActivityMessage |
MarketDataMessage | MarketCandleMessage | MarketHealthMessage |
KillCheckMessage | KillTriggerMessage | SystemCommandMessage |
AuthorizedCommandMessage | SecurityEventMessage | NetHealthMessage |
ExecHealthMessage | AnalyticsSnapshotMessage | PerformanceSnapshotMessage |
BuzzMessage<any>;

```

If you want, next I can provide a `decisionChainReducer.ts` that:

- maintains `signalMap` + `intentMap`

- upserts attempts by `client_order_id`
- handles out-of-order events + retries
- outputs a sorted `DecisionChainItem[]` for your Status page.

Provide the decision chain reducer

```
// decisionChainReducer.ts // Hivenance / SwarmOS – Decision Chain Builder
// (UI Reducer) // ===== //
// Consumes Buzz events and produces UI-ready DecisionChainItem[]. // // Goals:
// - intent-centric joining (primary key: intent_id) // - supports out-of-
// order delivery (request/execution may arrive before others) // - supports
// retries (attempt A/B/C) by client_order_id // - idempotent-ish (dedupe by
// buzz.id + per-attempt status signature) // // Dependencies: buzz-types.ts
// (interfaces) import { AnyBuzzMessage, StrategySignalMessage,
CoordinatorDecisionMessage, TradeRequestMessage, TradeExecutionMessage,
IntentStateMessage, DecisionChainItem, DecisionChainAttempt,
ExecutionStatus, ChainState, Venue, OrderType, } from "../buzz-types"; /* ---
----- * Types / State * ----- */
export interface DecisionChainState { // strategy signals keyed by signal_id
signalById: Record<string, StrategySignalMessage>; // decision chain items
keyed by intent_id chainById: Record<string, DecisionChainItem>; //
recent event ids for dedupe (simple bounded set) seenEventIds: string[]; //
max size for seenEventIds seenCap: number; } export type DecisionChainAction
= | { type: "INGEST_EVENT"; event: AnyBuzzMessage } | { type: "RESET" }; /*
----- * Helpers * ----- */
function nowTs(): number { return Date.now(); } function isSeen(state:
DecisionChainState, id: string): boolean { return
state.seenEventIds.includes(id); } function markSeen(state:
DecisionChainState, id: string): DecisionChainState { const next =
[...state.seenEventIds, id]; const capped = next.length > state.seenCap ?
next.slice(next.length - state.seenCap) : next; return { ...state,
seenEventIds: capped }; } function ensureChainItem( state:
DecisionChainState, intentId: string, defaults?: Partial<DecisionChainItem>
): DecisionChainItem { const existing = state.chainById[intentId]; if
(existing) return existing; // Minimal placeholder; will be enriched as
events arrive return { intentId, symbol: defaults?.symbol ?? "UNKNOWN",
side: defaults?.side ?? "BUY", strategy: defaults?.strategy, gate: { ts:
defaults?.gate?.ts ?? nowTs(), decision: defaults?.gate?.decision ??
"APPROVE", reason: defaults?.gate?.reason ?? "UNKNOWN", killState:
defaults?.gate?.killState, notes: defaults?.gate?.notes, policySnapshot:
defaults?.gate?.policySnapshot, }, attempts: [], chainState:
defaults?.chainState ?? "PENDING_REQUEST", }; } function upsertAttempt(
item: DecisionChainItem, clientId: string, patch:
Partial<DecisionChainAttempt> & { exec?:
```

```

Partial<DecisionChainAttempt["exec"]> } ): DecisionChainItem { const idx =
item.attempts.findIndex((a) => a.attemptId === clientOrderId); if (idx ===
-1) { // Create a new attempt with safe defaults const base:
DecisionChainAttempt = { attemptId: clientOrderId, venue: (patch.venue ??
"unknown") as Venue, orderType: (patch.orderType ?? "MARKET") as OrderType,
qty: patch.qty ?? 0, requestedTs: patch.requestedTs ?? nowTs(),
expireAfterSec: patch.expireAfterSec, exec: { lastStatus:
(patch.exec?.lastStatus ?? "RECEIVED") as ExecutionStatus, statuses:
patch.exec?.statuses ?? [], orderId: patch.exec?.orderId, filledQty:
patch.exec?.filledQty, avgPrice: patch.exec?.avgPrice, feesUsd:
patch.exec?.feesUsd, slippagePct: patch.exec?.slippagePct, latencyMs:
patch.exec?.latencyMs, error: patch.exec?.error, finalTs:
patch.exec?.finalTs, finalOutcome: patch.exec?.finalOutcome, }, }; return {
...item, attempts: [...item.attempts, base] }; } const existing =
item.attempts[idx]; const nextExec = { ...existing.exec, ...patch.exec, //
statuses handled separately (append/dedupe), so don't overwrite unless
provided explicitly statuses: patch.exec?.statuses ??
existing.exec.statuses, }; const nextAttempt: DecisionChainAttempt = {
...existing, ...patch, exec: nextExec, }; const nextAttempts =
[...item.attempts]; nextAttempts[idx] = nextAttempt; return { ...item,
attempts: nextAttempts }; } function appendStatus( attempt:
DecisionChainAttempt, ts: number, status: ExecutionStatus, details?:
Record<string, any> ): DecisionChainAttempt { // Dedupe by (status + ts) to
avoid spam from retries/replays. const exists =
attempt.exec.statuses.some((s) => s.status === status && s.ts === ts); if
(exists) return attempt; const statuses = [...attempt.exec.statuses, { ts,
status, details }].sort((a, b) => a.ts - b.ts); const lastStatus = status;
return { ...attempt, exec: { ...attempt.exec, statuses, lastStatus, }, }; }
function deriveChainState(item: DecisionChainItem): ChainState { // VETO is
terminal and has priority if (item.gate?.decision === "VETO") return
"VETOED"; // No attempts yet -> waiting to request/execute if
(!item.attempts.length) return "PENDING_REQUEST"; // Find the most recent
attempt by requestedTs const latest = [...item.attempts].sort((a, b) =>
(a.requestedTs ?? 0) - (b.requestedTs ?? 0)).pop(); if (!latest) return
"PENDING_REQUEST"; const s = latest.exec.lastStatus; if (s === "FILLED")
return "FILLED"; if (s === "PARTIAL") return "PARTIAL"; if (s === "PLACED"
|| s === "RECEIVED") return "PLACED"; if (s === "CANCELED") { // If there
exists a later attempt still active, don't mark canceled const anyActive =
item.attempts.some( (a) => ["RECEIVED", "PLACED",
"PARTIAL"].includes(a.exec.lastStatus) ); return anyActive ? "PLACED" :
"CANCELED"; } if (["REJECTED", "EXPIRED", "ERROR",
"PRECHECK_FAILED"].includes(s)) { // If there is another attempt after a
failed one, chain may still be in progress. const anyActive =
item.attempts.some( (a) => ["RECEIVED", "PLACED",
"PARTIAL"].includes(a.exec.lastStatus) ); return anyActive ? "PLACED" :
"FAILED"; } return "PENDING_REQUEST"; } function sortDecisionChains(chains:
DecisionChainItem[]): DecisionChainItem[] { // Sort by gate ts desc (latest
decisions first) return [...chains].sort((a, b) => (b.gate?.ts ?? 0) -
(a.gate?.ts ?? 0)); } function attachSignalIfAvailable( state:
DecisionChainState, item: DecisionChainItem, signalId?: string ):

```

```

DecisionChainItem { if (!signalId) return item; const sig =
state.signalById[signalId]; if (!sig) return item; return { ...item,
strategy: item.strategy ?? sig.payload.strategy, signal: { signalId:
sig.payload.signal_id, ts: sig.buzz.ts, action: sig.payload.action,
confidence: sig.payload.confidence, featuresSnapshot:
sig.payload.features_snapshot, }, }; } /* ----- *
Event handlers * ----- */ function handleStrategySignal(
state: DecisionChainState, evt: StrategySignalMessage ): DecisionChainState
{ const signalId = evt.payload.signal_id; return { ...state, signalById: {
...state.signalById, [signalId]: evt }, }; } function
handleCoordinatorDecision( state: DecisionChainState, evt:
CoordinatorDecisionMessage ): DecisionChainState { const intentId =
evt.payload.intent_id; let item = ensureChainItem(state, intentId, { symbol:
evt.payload.symbol, side: evt.payload.action, strategy: undefined, gate: {
ts: evt.buzz.ts, decision: evt.payload.decision, reason: evt.payload.reason,
killState: evt.payload.kill_state_at_decision, notes: evt.payload.notes,
policySnapshot: evt.payload.policy_snapshot, }, }); // Overwrite gate with
latest coordinator decision (by ts) if (!item.gate || evt.buzz.ts >=
item.gate.ts) { item = { ...item, symbol: evt.payload.symbol ?? item.symbol,
side: evt.payload.action ?? item.side, gate: { ts: evt.buzz.ts, decision:
evt.payload.decision, reason: evt.payload.reason, killState:
evt.payload.kill_state_at_decision, notes: evt.payload.notes,
policySnapshot: evt.payload.policy_snapshot, }, }; } item =
attachSignalIfAvailable(state, item, evt.payload.signal_id); item = {
...item, chainState: deriveChainState(item) }; return { ...state,
chainById: { ...state.chainById, [intentId]: item }, }; }
function handleTradeRequest( state: DecisionChainState, evt:
TradeRequestMessage ): DecisionChainState { const { intent_id,
client_order_id } = evt.payload; let item = ensureChainItem(state,
intent_id, { symbol: evt.payload.symbol, side: evt.payload.side, strategy:
evt.payload.origin_strategy, }); item = upsertAttempt(item, client_order_id,
{ venue: evt.payload.venue, orderType: evt.payload.order_type, qty:
evt.payload.qty, requestedTs: evt.buzz.ts, expireAfterSec:
evt.payload.expire_after_sec, exec: { // If we already have later statuses
from execution, don't reset them. lastStatus: (item.attempts.find((a) =>
a.attemptId === client_order_id)?.exec.lastStatus ?? "RECEIVED") as
ExecutionStatus, }, }); item = { ...item, chainState: deriveChainState(item)
}; return { ...state, chainById: { ...state.chainById,
[intent_id]: item }, }; } function handleTradeExecution( state:
DecisionChainState, evt: TradeExecutionMessage ): DecisionChainState { const
p = evt.payload; let item = ensureChainItem(state, p.intent_id, { symbol:
p.symbol, side: "BUY", }); // Ensure attempt exists item =
upsertAttempt(item, p.client_order_id, { venue: p.venue, orderType:
"MARKET", qty: item.attempts.find((a) => a.attemptId ===
p.client_order_id)?.qty ?? 0, requestedTs: item.attempts.find((a) =>
a.attemptId === p.client_order_id)?.requestedTs ?? evt.buzz.ts, }); //
Update attempt execution timeline const idx = item.attempts.findIndex((a) =>
a.attemptId === p.client_order_id); if (idx >= 0) { let attempt =
item.attempts[idx]; attempt = appendStatus(attempt, evt.buzz.ts, p.status, {
filled_qty: p.filled_qty, avg_price: p.avg_price, fee_usd: p.fee_usd,

```

```

    slippage_pct: p.slippage_pct, latency_ms: p.latency_ms, error: p.error, });
    const finalStatuses: ExecutionStatus[] = ["FILLED", "CANCELED", "REJECTED",
    "EXPIRED", "ERROR", "PRECHECK_FAILED"]; const isFinal =
    finalStatuses.includes(p.status); // Patch summary fields attempt = {
    ...attempt, exec: { ...attempt.exec, lastStatus: p.status, orderId:
    p.order_id ?? attempt.exec.orderId, filledQty: p.filled_qty ??
    attempt.exec.filledQty, avgPrice: p.avg_price ?? attempt.exec.avgPrice,
    feesUsd: p.fee_usd ?? attempt.exec.feesUsd, slippagePct: p.slippage_pct ??
    attempt.exec.slippagePct, latencyMs: p.latency_ms ?? attempt.exec.latencyMs,
    error: p.error ?? attempt.exec.error, finalTs: isFinal ? evt.buzz.ts :
    attempt.exec.finalTs, finalOutcome: isFinal ? (p.status as any) :
    attempt.exec.finalOutcome, statuses: attempt.exec.statuses, }, }; // Replace
    attempt const attempts = [...item.attempts]; attempts[idx] = attempt; item =
    { ...item, attempts }; } item = { ...item, chainState:
    deriveChainState(item) }; return { ...state, chainById: {
    ...state.chainById, [p.intent_id]: item }, }; } function
    handleIntentState( state: DecisionChainState, evt: IntentStateMessage ):
    DecisionChainState { const p = evt.payload; let item =
    ensureChainItem(state, p.intent_id, { symbol: p.symbol, }); // If intent is
    done, we can mark a final outcome in the latest attempt if present. if
    (p.state === "INTENT_DONE" && p.final_outcome && p.client_order_id) { item =
    upsertAttempt(item, p.client_order_id, { exec: { lastStatus:
    (p.final_outcome as ExecutionStatus) as ExecutionStatus, finalOutcome:
    p.final_outcome as any, finalTs: evt.buzz.ts, }, }); } item = { ...item,
    chainState: deriveChainState(item) }; return { ...state, chainById: {
    ...state.chainById, [p.intent_id]: item }, }; } /* -----
    ----- * Reducer * ----- */ export function
    createInitialDecisionChainState( opts?: Partial<Pick<DecisionChainState,
    "seenCap">> ): DecisionChainState { return { signalById: {},
    chainById: {}, seenEventIds: [], seenCap: opts?.seenCap ?? 2000, }; }
    export function decisionChainReducer( state: DecisionChainState, action:
    DecisionChainAction ): DecisionChainState { switch (action.type) { case
    "RESET": return createInitialDecisionChainState({ seenCap: state.seenCap });
    case "INGEST_EVENT": { const evt = action.event; const evtId =
    evt?.buzz?.id; if (evtId && isSeen(state, evtId)) return state; let next =
    evtId ? markSeen(state, evtId) : state; // Route by type switch
    (evt.buzz.type) { case "buzz.strategy.signal": return
    handleStrategySignal(next, evt as StrategySignalMessage); case
    "buzz.coordinator.decision": return handleCoordinatorDecision(next, evt as
    CoordinatorDecisionMessage); case "buzz.trade.request": return
    handleTradeRequest(next, evt as TradeRequestMessage); case
    "buzz.trade.execution": return handleTradeExecution(next, evt as
    TradeExecutionMessage); case "buzz.intent.state": return
    handleIntentState(next, evt as IntentStateMessage); default: return next; }
    } default: return state; } } /* ----- * Selectors * --
    ----- */ export function selectDecisionChainList(state:
    DecisionChainState, limit = 50): DecisionChainItem[] { const list =
    Object.values(state.chainById); const sorted =
    sortDecisionChains(list); return sorted.slice(0, limit); } /* -----
    ----- * Notes / Extensions * ----- * 1) If you

```

want to show VETO rows that have no intent_id yet: * - You can require coordinator to always generate intent_id even for veto. * * 2) If you want to attach Strategy signals without signal_id: * - You can add "proposed_intent_key" = `\${symbol}:\${action}:\${tsBucket}` * but that gets fuzzy. signal_id is strongly recommended. * * 3) If you want stronger dedupe on statuses: * - Use buzz.id + status signature LRU per attempt. */